Reinsurance Recon Utility

Product Case Study — Detailed Reference

Role: Solo Product Owner & Builder
Context: Big 4 Consulting — Global Reinsurance Finance Transformation Program
Stack: Streamlit · Pandas · PySpark · SQL
Status: Live in production · Expanding to additional reinsurance projects

Details anonymized to protect client confidentiality.

1. Executive Summary

During a large-scale reinsurance Finance Transformation program, a critical gap was identified in the processing validation cycle — reconciliation of ceded reinsurance amounts against treaty rules was being done manually in Excel, taking 2 working days per monthly cycle, and had become a dependency on technical QA resource rather than an independently operable BAU process.

A self-serve Reinsurance Recon Utility was conceived, defined, and built — a hard gate validation tool that sits between the reinsurance engine and downstream systems (subledger, GL, reporting, settlement) — ensuring cession accuracy is confirmed against treaty rules before any financial data is released downstream.

Through three iterations driven by user feedback and adoption barriers, the tool evolved from a SQL script to a fully self-serve Streamlit application operable by non-technical BAU analysts with a simple set of clicks.

Metric	Result
Recon cycle time	2 working days → 15 minutes (99.6% reduction)
BAU dependency	Technical QA dependency fully eliminated
Scale	Millions of transactions, billions in financial impact per cycle
Adoption	Expanding to additional reinsurance projects firm-wide

2. The Problem

Context

A large global insurer was undergoing an enterprise-wide Finance Transformation program — modernizing multiple platforms to enhance month-end close, eliminate reporting lag, drive automation, and standardize group companies onto a common GL. The RI Transformation stream replaced disparate homegrown systems and manual processes with a unified, automated, globally standardized solution.

At the heart of this transformation was a reinsurance engine — responsible for processing inbound gross transaction data (premiums, commissions, claims, reserves) and applying treaty rules to produce ceded, retained, and error-classified output. This output feeds directly into downstream subledgers, GL, reporting layers, and reinsurer settlement — making accuracy non-negotiable.

The Gap

Every production cycle, someone had to answer one question: *did the reinsurance engine apply the treaty rules correctly?*

The answer required reconciling actual engine output against expected cession amounts — calculated manually from treaty parameters including attachment criteria, ceding percentages, and contract configuration — for every policy, every cycle. This was being done in Excel.

- Manual recon of one month's data took **2 full working days** per cycle
- BAU analysts regularly offloaded recon work to the QA team due to volume and complexity
- The QA team became the de facto recon resource on top of their primary validation responsibilities
- Errors were caught late, after significant analyst effort had already been spent
- At scale — millions of transactions per run, billions in financial impact — this was a critical control gap

Why It Mattered

Risk	Description
Financial Misstatement	Incorrect ceded amounts appear in financial statements — regulatory and audit exposure at g
Treaty Breach	Policies ceded outside attachment criteria violate contract terms and can void reinsurer recoveries
Audit & Compliance	Manual recon with no automated control is a flagged weakness in internal and external audit reviews
Operational Cost	2 working days of senior analyst time per cycle, every month, across multiple active treaties

---

3. Opportunity Validation

Before building anything, every product opportunity must pass three checks. This one passed all three.

Check 1 — Strategic Fit ✓

The utility sat squarely within the RI Transformation stream's mandate — replacing manual, technically-dependent processes with automated, standardized solutions operable by BAU teams globally. It directly served four Finance Transformation Program objectives: month-end close enhancement, reporting lag elimination, automation of manual processes, and strengthening analytical capabilities for business partners.

Check 2 — User Value ✓

Three user groups were directly impacted:

BAU Operations Analysts

Non-technical users responsible for running monthly reinsurance processing cycles. Currently spending 2 working days per cycle on manual Excel recon — interpreting per-policy attachment criteria individually, with no automated support. Pain was high severity: the work was slow, error-prone, and had spilled over into QA team capacity.

QA / Data Engineers

Technical users responsible for validation — absorbing BAU recon overflow on top of their primary responsibilities. The dependency was unsustainable and misallocated skilled technical resource to manual operational work.

Product Owners / Delivery Leads

Responsible for signing off on downstream release covering billions in financial impact. Without an automated validation gate, sign-off relied on manual recon outputs that were time-consuming to produce and difficult to audit. The utility gave POs structured, automated evidence of cession accuracy — replacing subjective manual checks with a repeatable, documented control.

Check 3 — Business Value ✓

Dimension	Content
Qualitative	Stronger automated controls; reduced audit risk; elimination of manual recon as a control weakness; faster month
Quantitative	99.6% reduction in recon cycle time (2 days → 15 minutes); QA recon dependency eliminated; measurable impro
Non-Goals	Will NOT be judged on reinsurance engine performance, upstream data quality, or broader Finance Transformati

A structured Manager Briefing was completed before any build work began — documenting hypotheses on strategic fit, user value, and business value across all stakeholders. See Appendix A.

---

4. The Product

What It Does

The Reinsurance Recon Utility is a self-serve validation application that automatically reconciles actual reinsurance engine output against expected cession amounts — derived from treaty rules — acting as a validation gate before any financial data is released to downstream systems.

In plain terms: it answers the question *"did the reinsurance engine apply the treaty rules correctly?"* — automatically, in 15 minutes, for every policy across every treaty in a production cycle.

How It Works

Input	What It Contains
Treaty Report / Coding Sheet	Contract configuration — attachment criteria, ceding percentages, contract name, retention rule
Inbound File	Gross transaction data from source systems — premiums, commissions, claims, reserves
Reinsurance Engine Output	Actual processed data — ceded amounts, retained amounts, error-classified transactions

Using the treaty report as the source of truth, the utility calculates **expected cession** for every policy — applying attachment criteria and ceding percentages independently. It then compares expected against actual engine output across four dimensions:

- Correctly ceded transactions
- Transactions present in error queue — flagged with reason
- Transactions incorrectly retained — flagged for investigation
- Cession amount mismatches — actual vs expected variance surfaced

Architecture

Source Systems (gross transaction data)
↓
Reinsurance Engine (applies treaty rules → produces ceded / retained / error output)

↓

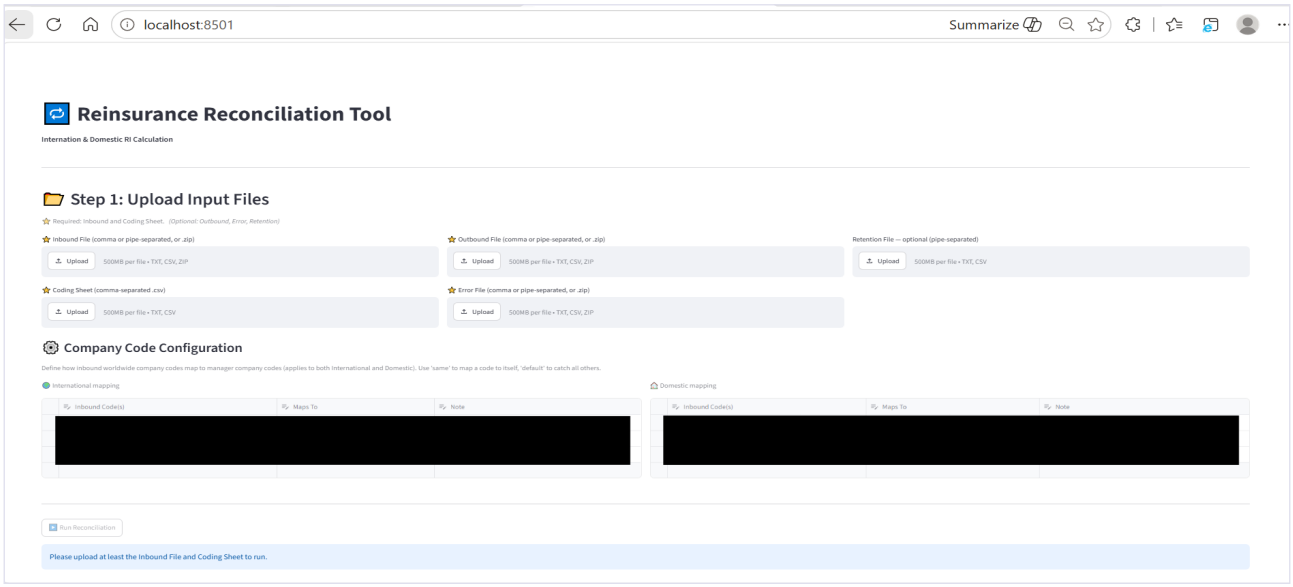
RECON UTILITY — Treaty Rules → Expected vs Actual → Validated

↓ (BAU reviews output and releases to downstream after validation passes)

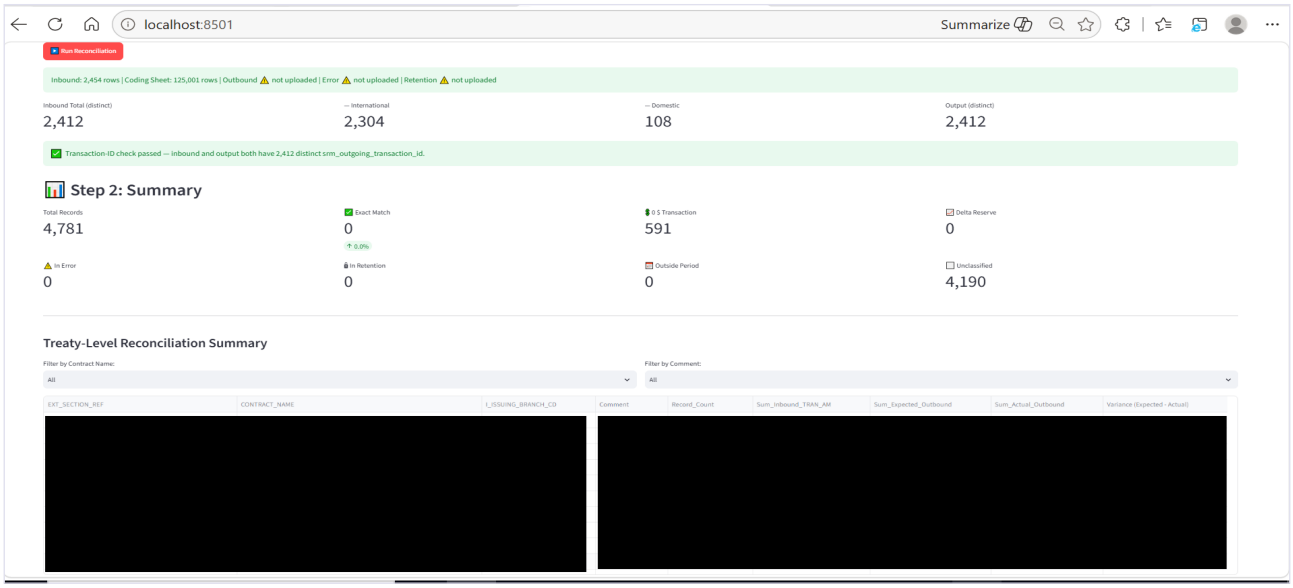
Downstream: Subledger → GL → Reporting → Settlement

Application Screenshots

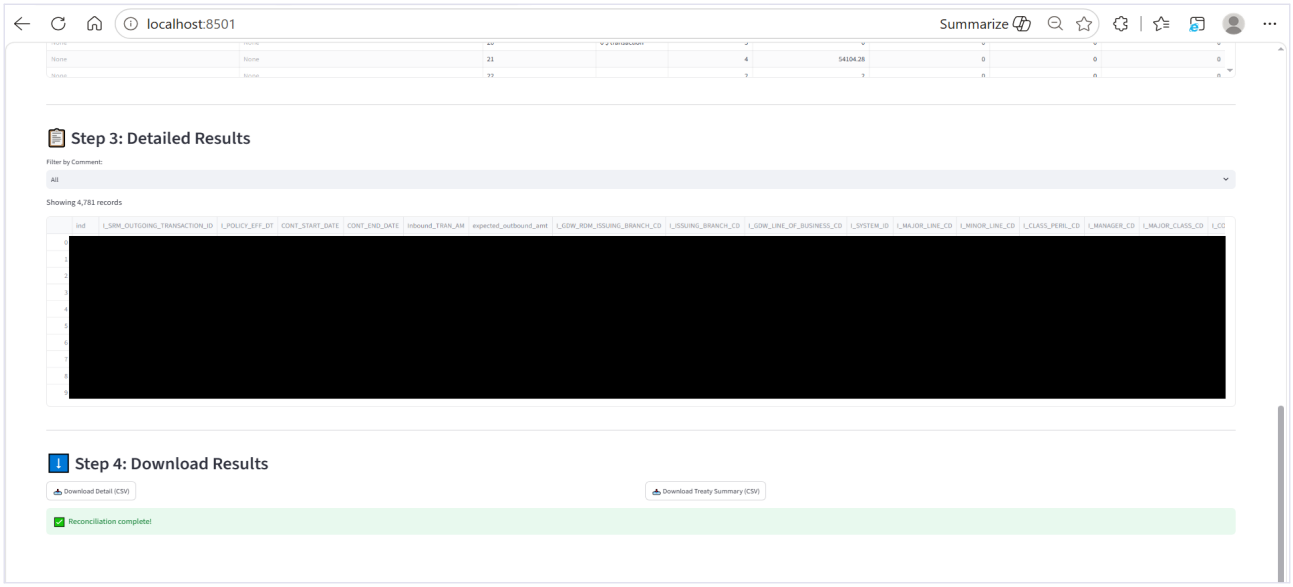
The utility is built in Streamlit — a browser-based interface requiring no technical prerequisites. A BAU analyst runs a full recon cycle through a simple sequence of file uploads and clicks.



Step 1 — Upload interface



Step 2 — Recon summary dashboard



Step 3 — Detailed results & download

5. Key Product Decisions

Decision 1 — SQL → PySpark → Streamlit: the full evolution

Recon began as manual SQL queries. Beyond requiring direct database access and technical execution, SQL introduced significant operational friction at scale — input files had to be uploaded to a database first, file format compatibility was inconsistent, large files took hours to load, and computation on high transaction volumes was slow. Each run was a multi-step manual process dependent on technical resource.

PySpark was the natural next step — eliminating the database dependency and handling large-scale data processing efficiently. But it introduced a new problem: BAU analysts could not run it. Command-line execution, environment dependencies, and script management created a hard technical barrier. The tool worked — but only if a data engineer ran it.

Streamlit eliminated the barrier entirely. A browser-based interface with file upload and click-based execution required zero technical prerequisites. BAU analysts could run a full recon cycle independently — no database uploads, no format issues, no script execution.

Decision 2 — Process-enforced validation gate, not system-enforced block

The utility operates as a process control gate — BAU runs recon first, reviews output, and only releases files to downstream systems after validation passes. Data does not flow automatically — release is a deliberate human action taken only after the utility confirms cession accuracy. A manual release step after automated validation gives BAU and delivery leads a documented checkpoint — a moment of explicit sign-off before billion-dollar financial data moves downstream. Automating the release itself would have removed that accountability layer.

Decision 3 — Treaty rules as single source of truth

Expected cession is calculated directly from the treaty report — not from separate configuration or hardcoded logic. This means the utility stays accurate as treaty terms change, without requiring code updates. The treaty report is the product's logic engine.

Decision 4 — Expanding output beyond mismatch flagging

Early versions only flagged cession amount mismatches. Stakeholder feedback revealed BAU needed visibility into why transactions went to error or retention — not just that they did. Output was expanded to surface error-queue presence and retention analysis alongside mismatch flagging. This came directly from the demo session with BAU and business stakeholders.

Decision 5 — Single file handling for international and domestic transactions

Reinsurance processing rules differ between international and domestic transactions — attachment criteria, ceding structures, and treaty configurations vary significantly. An early design limitation required separate files for each transaction type. Following stakeholder feedback during demo sessions, the utility logic was redesigned to handle both within a single input file — automatically identifying and applying the correct treaty rules per transaction type. This eliminated a manual pre-processing step for BAU and reduced recon setup time significantly.

Decision 6 — Dynamic configurable parameters on the Streamlit screen

Certain recon parameters vary across production cycles and treaty types. Rather than hardcoding these values or requiring code changes for each variation, key parameters were surfaced as configurable inputs directly on the Streamlit interface. BAU analysts can adjust parameters per cycle without any developer involvement — making the utility genuinely self-sufficient and eliminating a maintenance dependency on technical resource.

Decision 7 — Recon summary surfaced directly on screen

Rather than requiring BAU to download and interpret detailed output files, a high-level recon summary is presented directly on the Streamlit screen at the end of each run — showing overall match status, transaction counts by category (ceded, retained, error), and key mismatch flags at a glance. Detailed drill-down remains available in the output files for investigation. This two-layer output design — summary on screen, detail in file — matched how BAU and delivery leads actually consumed results: quick status check first, deep dive only when needed.

---

6. The Iteration Story

> ***Technical correctness is not the same as product success. A tool that works perfectly but cannot be used by its intended users has failed its purpose.***

V1 — SQL Scripts

Goal: Automate manual Excel recon

The first version replaced Excel with SQL queries — faster, more accurate, and handling higher transaction volumes. But it required database access and SQL knowledge. File uploads to the database were time-consuming, file format sanitization was difficult, and computation on large datasets was slow. Every production cycle still required technical resource involvement.

- **What broke:** Data uploads to DB were time-consuming; file format sanitization was difficult; computation on large datasets was slow. BAU still could not run it without a data engineer.
- **What it revealed:** The real problem was not just recon accuracy. It was recon accessibility.

V2 — PySpark Script

Goal: Eliminate database dependency, handle scale

PySpark removed the database upload requirement entirely — files processed directly, handling millions of transactions efficiently. Technically the best version yet. But adoption failed for the same reason as V1.

- **What broke:** PySpark environment setup was a barrier; command-line execution was too technical for BAU users to run independently.
- **What it revealed:** The barrier was not the tool's logic — it was the tool's interface. The recon engine was right. The delivery mechanism was wrong.

V3 — Streamlit Application

Goal: Full BAU self-sufficiency, zero technical prerequisites

Streamlit repackaged the same recon logic into a browser-based interface — file upload, configurable parameters, click to run, summary on screen, detailed output for download. No database. No command line. No environment setup. BAU analysts ran it independently from day one.

- BAU self-sufficient across all production cycles
- QA team recon dependency eliminated
- Recon cycle time: 2 working days → 15 minutes
- POs signing off on downstream release with automated, documented evidence
- Tool now being adopted by additional reinsurance projects within the firm

> ***The problem to solve was no longer recon accuracy — that was solved in V1. The unsolved problem was making accurate recon accessible to the people who needed to run it.***

Iteration Summary

Version	Stack	Problem Solved	Problem Revealed
V1	SQL	Recon accuracy	DB uploads slow; format issues; compute-heavy; BAU dependent
V2	PySpark	Scale and DB dependency	PySpark setup and CLI execution inaccessible to non-technical BA
V3	Streamlit + Pandas	Full BAU self-sufficiency	—

The requirements document told me what the tool needed to do. The users told me what the tool needed to be.

---

7. Stakeholder Collaboration

Building the utility was only half the work. The recon logic, output design, and interface decisions were shaped through active collaboration with three stakeholder groups — each with different needs, different technical levels, and different definitions of what "good" looked like.

Business Analysts — Defining the Logic

Working closely with the BA team, treaty parameters were mapped to expected cession calculations — confirming edge cases, clarifying how international and domestic transactions were handled differently under the

same treaty, and validating that the utility's expected output matched what the business considered correct. This collaboration was the foundation of the utility's accuracy. Without it, the tool would have been technically functional but logically wrong.

What it produced:

- Confirmed recon logic per treaty type
- Edge case handling for attachment criteria variations
- Correct treatment of international vs domestic transactions within the same processing run

BAU Operations Analysts — Shaping the Interface

BAU analysts were the primary end users — and the group whose feedback most directly shaped the product. Demo sessions revealed gaps that were not visible during build: the need to handle international and domestic transactions in a single file, the need for configurable parameters on screen, and the need for a summary view on screen. Each of these became a product decision. None were anticipated during initial build.

What it produced:

- Single file handling for mixed transaction types
- Dynamic configurable parameters — no developer involvement required per cycle
- Two-layer output design: summary on screen, detail in downloadable file

Product Owners / Delivery Leads — Building Confidence

POs needed confidence that data released to downstream systems was accurate — and that confidence needed to be documented, not just asserted. The utility gave POs a structured, repeatable validation output — the same checks, the same logic, every cycle — replacing subjective manual review with automated, documented evidence of cession accuracy. Results and output were presented directly to POs and business stakeholders.

What it produced:

- Structured, repeatable validation evidence per production cycle
- Documented checkpoint before every downstream release decision
- Stakeholder confidence in the data gate — replacing manual judgment with automated control

> ***The requirements document told me what the tool needed to do. The users told me what the tool needed to be.***

---

8. Outcomes

Quantitative Results

Metric	Before	After	Impact
Recon cycle time	2 working days	15 minutes	99.6% reduction
BAU technical dependency	QA team runs recon	BAU runs independently	Fully eliminated
Recon method	Manual Excel	Automated Streamlit app	Zero manual calculation

Release confidence	Subjective manual review	Automated documented evidence	Structured control
Tool adoption	Single project	Expanding firm-wide	Organic internal growth

Qualitative Results

For BAU Analysts

The 2-day manual recon cycle — requiring per-policy interpretation of attachment criteria across millions of transactions — was replaced by a process any BAU analyst could run independently in 15 minutes. The cognitive load of manually interpreting treaty rules per policy, per cycle, was eliminated entirely. BAU analysts moved from being blocked by recon to being in control of it.

For QA / Data Engineers

Recon overflow work that had been absorbing QA capacity every production cycle was eliminated. Technical resource was freed to focus on actual validation responsibilities — the work it was hired to do — rather than operational recon support.

For Product Owners / Delivery Leads

Every downstream release decision — covering millions of transactions and billions in financial impact — is now backed by automated, documented validation evidence rather than manual judgment. POs sign off with confidence, not with hope.

For the Firm

The utility is no longer a single-project tool. Other reinsurance projects within the firm are in the process of adopting it — recognising the same manual recon problem exists across engagements and that the utility solves it without customisation. What started as a solution to one project's operational gap is becoming a reusable internal product.

> ***99.6% reduction in recon cycle time — across a system processing millions of transactions covering billions in financial impact, every production cycle. That number exists because the right problem was identified, the right users were involved, and the tool was iterated until it actually worked for the people who needed to use it — not just for the person who built it.***

---

9. What I Would Do Differently

1. Involve BAU users from V1, not V3

The usability gap that killed V1 and V2 adoption was entirely predictable — if a BAU analyst had been observed during V1, the interface problem would have surfaced immediately. Instead, two versions were built optimising for technical correctness before the real barrier was discovered. In hindsight, the right approach was to define the user's technical constraints *before* choosing the delivery mechanism — not after two iterations of building the wrong interface for the right logic.

What I'd do differently: Run a single user observation session with a BAU analyst before writing a line of V1 code. The SQL → PySpark → Streamlit journey was necessary learning — but it could have been compressed significantly.

2. Define success metrics before build, not after

The 99.6% cycle time reduction is a compelling outcome — but it was measured retrospectively. There was no formal baseline captured at the start: no documented record of exactly how long manual recon took, which steps consumed the most time, or what error rate the manual process produced. The outcome is real and the improvement is significant — but a formally documented baseline would have made the case for the utility stronger earlier and accelerated stakeholder buy-in.

What I'd do differently: Document the baseline explicitly before building anything — time per step, error rate, resource hours per cycle. Make the problem measurable before making the solution.

3. Structure the demo sessions as formal user feedback rounds

The demo sessions with BAU and business stakeholders were valuable — several key product decisions came directly from them. But they were run informally, without a structured agenda or pre-defined questions. Feedback was collected conversationally rather than systematically. As a result, some requirements emerged late.

What I'd do differently: Treat every demo session as a structured feedback round — define what decisions need input before the session, prepare specific scenarios to walk through, and capture observations systematically rather than relying on open conversation.

4. Document product decisions in real time

The product decisions captured in this case study were reconstructed from memory and project artifacts after the fact. During the build, decisions were made and implemented without formal documentation of the alternatives considered, the reasoning applied, or the tradeoffs accepted. For a portfolio piece, this creates a reconstruction challenge. For a real product team, it creates an institutional knowledge gap.

What I'd do differently: Maintain a lightweight decision log throughout — one line per decision: what was considered, what was chosen, why.

---

10. PM Learnings

1. The user's constraint is part of the product requirement

Before this project, requirements were defined in terms of what the tool needed to do — the logic, the calculations, the output. V1 and V2 were both technically correct by that definition. What I learned: the user's ability to operate the tool is as much a requirement as the tool's logic. Usability is not a nice-to-have layer on top of functionality — it is functionality. This reframed how I think about requirements entirely: every requirement now has two dimensions — what the product does and who can do it without help.

2. The real problem is rarely the stated problem

The stated problem was: recon is inaccurate and slow. V1 solved both. By the stated problem definition, V1 was a success. But adoption failed because the real problem — never articulated at the start — was that recon required technical resource that shouldn't be needed for an operational BAU task. The PM's job is to keep asking why until the root cause is clear — and to validate that the root cause, not just the symptom, is what's being solved.

3. Users show you what they need — they rarely tell you

The most important product decisions in this build — single file handling, configurable parameters, summary on screen — were never in any requirements document. They emerged from watching BAU analysts interact with the tool during demo sessions. No stakeholder said 'I need a summary on screen.' What was observed was: every BAU analyst's first action after a recon run was to look for an overall status. Structured observation during demo sessions is more valuable than open-ended feedback collection.

4. Iteration is not failure — premature handoff is

Moving from SQL to PySpark to Streamlit could be read as two failures before a success. Each version answered a question the previous one couldn't. The failure mode wasn't iterating — it was that V2 was handed to BAU before V3's question had been asked. Before handing any product to its intended users, explicitly ask: can this person use this without help? If the answer is no, the product is not ready — regardless of how technically correct it is.

5. A product that spreads organically has found its market

The utility was built for one project. Other reinsurance projects within the firm are now adopting it — not because of a rollout plan or a mandate, but because the same problem exists across engagements and the tool solves it without customisation. Organic adoption is the most honest product metric. It happens when a product solves a real problem well enough that users recommend it without being asked. Everything else — usage metrics, stakeholder sign-off, positive demo feedback — can be manufactured. Organic spread cannot.
